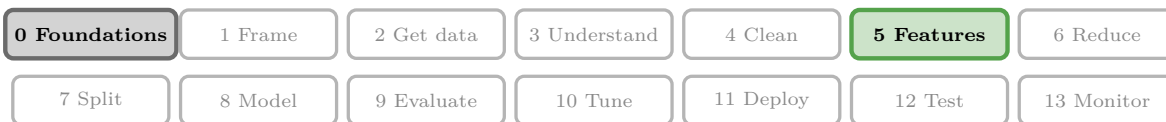


Vectors and Feature Vectors

Contents

1. Overview	2
2. What linear algebra is	3
3. Why ML needs linear algebra	3
3.1 Working in high dimensions	3
3.2 Representing data	3
4. What a vector is	4
4.1 A point in 2D	4
4.2 From 2D to n dimensions	4
4.3 Dimension and notation	4
5. Feature vectors	5
5.1 The iris flowers	5
5.2 Feature vectors hold only numbers	6
6. Text as vectors: a movie recommender	7
6.1 The problem	7
6.2 Bag of words	7
6.3 Recommending by distance	8
7. Row vectors and column vectors	9
8. Summary	11
9. Key terms	11

Where this fits: Pipeline map steps 0 (Foundations), 5 (Engineer features). See the [Course map](#).



- **Builds on:** Tensors ([Note 11](#)).
- **Leads to:** Vector magnitude, distance and scalar operations ([Note 361](#)); Dot product ([Note 362](#)).
- **Compare with:** One-hot encoding ([Note 27](#)).

1. Overview

Key point: A vector is a point in a coordinate space, written as a list of numbers; in ML, every data point is turned into such a vector, called a feature vector.

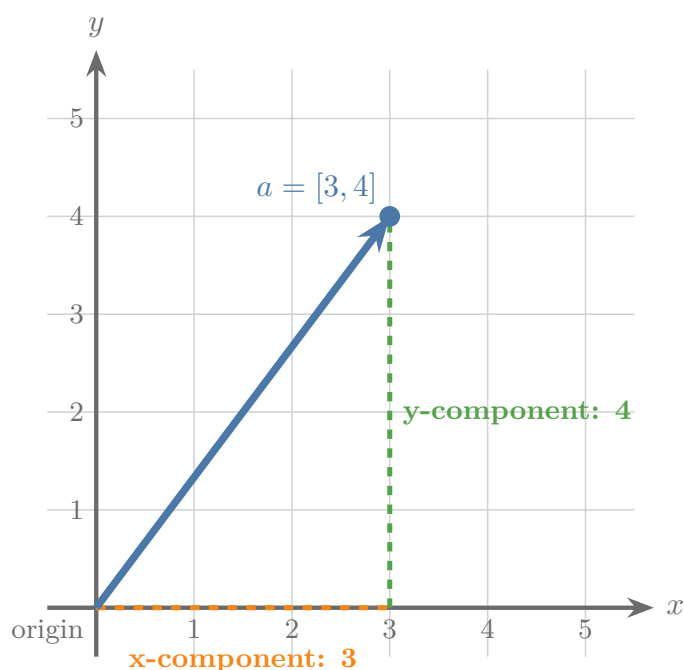


Figure 1: The vector $a = [3, 4]$: 3 units along x , then 4 units along y

Figure 1 shows the simplest vector there is: a point in a 2D plane, reached by moving 3 units along x and 4 along y . This Note starts with what linear algebra is and why ML needs it. It then defines a vector, shows how every row of a dataset (even a row of text) becomes a vector, and ends with the two ways of writing a vector: as a row or as a column.

2. What linear algebra is

Key point: Linear algebra is the branch of mathematics that studies linear equations and the objects used to write them: scalars, vectors, matrices and tensors.

Linear algebra is the branch of mathematics that deals with linear systems: sets of equations in which every variable appears only multiplied by a number and added up, such as $2x + 3y = 7$. We meet its first flavour in school, when we solve two equations in two unknowns.

It is one of the most foundational subjects in mathematics. Computer science, engineering, physics and economics all use it. It studies four kinds of objects, each introduced in the [tensors Note](#):

- **Scalars:** single numbers, such as 2, 5 or -6.
- **Vectors:** 1D collections of numbers.
- **Matrices:** 2D collections of numbers.
- **Tensors:** collections in any number of dimensions.

3. Why ML needs linear algebra

Key point: Linear algebra lets us work in any number of dimensions as easily as in two, and it lets us represent every kind of data as numbers.

3.1 Working in high dimensions

Key point: What holds in 2D and 3D holds in any number of dimensions; linear algebra gives us the eyes we lack beyond 3D.

We can picture at most three dimensions. ML data rarely stops there.

Take predicting salary from years of experience. The data is 2D, and linear regression draws a line through it. Add a column for education marks and the data becomes 3D: we can still picture the plane that fits it (see the [multiple linear regression Note](#)). Add one more column, or ten, and we can no longer picture anything.

Linear algebra solves this. Its ideas are written so that whatever is true in 2D and 3D stays true in n dimensions: the line becomes a plane, and the plane becomes a hyperplane, with the same kind of equation. Datasets with 5,000 columns are common, and linear algebra handles them the same way it handles two.

3.2 Representing data

Key point: Tables, text, images and video can all be written as vectors, matrices and tensors, and ML algorithms only accept such objects.

ML meets many kinds of data: tables, text, images, even video. An algorithm can only learn from numbers, so each kind must first be written as a vector, matrix or tensor. Linear algebra

is what makes this representation possible; once the data is in that form, any ML algorithm can take it in and make predictions.

Linear algebra also suits the hardware. Its operations apply the same step to many numbers at once, which is exactly what CPUs and especially GPUs do well. This is why deep learning, which runs almost entirely on GPUs, is built on linear algebra.

4. What a vector is

Key point: A vector is a point in a coordinate system, described by its components, one per axis; the number of components is its dimension.

4.1 A point in 2D

Key point: The point $(3, 4)$ means “3 units along x , then 4 units along y ”: that is the vector $[3, 4]$.

Start with 2D coordinate geometry from school. In Figure 1, the point $(3, 4)$ means: from the origin, move 3 units in the x direction, then 4 units in the y direction. We can treat this point as a **vector** and write it $a = [3, 4]$, drawn as an arrow from the origin (its tail) to the point (its head).

The individual numbers of a vector are its **components**. Here 3 is the x -component and 4 is the y -component.

4.2 From 2D to n dimensions

Key point: A 3D vector has three components; an n -dimensional vector has n .

Linear algebra always lets us start in small dimensions and grow:

- **3D:** a point in a 3D coordinate system is a vector with three components, $a = [a_1, a_2, a_3]$.
- **n D:** a point in an n -dimensional space is a vector with n components, $x = [x_1, x_2, \dots, x_n]$.

We cannot draw the n -dimensional case, but nothing about the vector changes. So, in short: a vector is a point in a coordinate system.

4.3 Dimension and notation

Key point: The dimension of a vector is the dimension of the space it lives in, which is its number of components.

The **dimension** of a vector is the dimension of the coordinate system it lives in. A vector in 2D space has dimension 2; in 3D, dimension 3; in n -dimensional space, dimension n . This is the same “dimension of a vector” as in the [tensors Note](#) (section 5): the number of items in it, not the number of axes of the tensor.

In ML we usually write a vector in square brackets with its components separated by commas: $[x_1, x_2, \dots, x_n]$ is an n -dimensional vector.

5. Feature vectors

Key point: The input columns of one row form a feature vector; an ML model takes feature vectors in and gives predictions out.

5.1 The iris flowers

Key point: Each of the 150 iris flowers becomes a 4-dimensional vector of its measurements.

The iris dataset, a classic first ML dataset, describes 150 flowers with five columns. Four are measurements in centimetres: sepal length, sepal width, petal length and petal width. The fifth is the species: setosa, versicolor or virginica.

The four measurements are the input columns; the species is the target column (see the [types of ML Note](#)), because it is what we want to predict from the inputs. The first flower in the dataset has measurements 5.1, 3.5, 1.4 and 0.2, and is a setosa.

A **feature vector** is the vector formed by the input values of one data point. For the first flower it is

$$x = [5.1, 3.5, 1.4, 0.2]$$

a vector in 4-dimensional space. To ask a model for this flower's species, we hand it this vector; the model works on it and returns a prediction, here “setosa”. Every flower becomes its own feature vector, so the dataset is 150 feature vectors.

150 iris flowers = 150 feature vectors in 3D

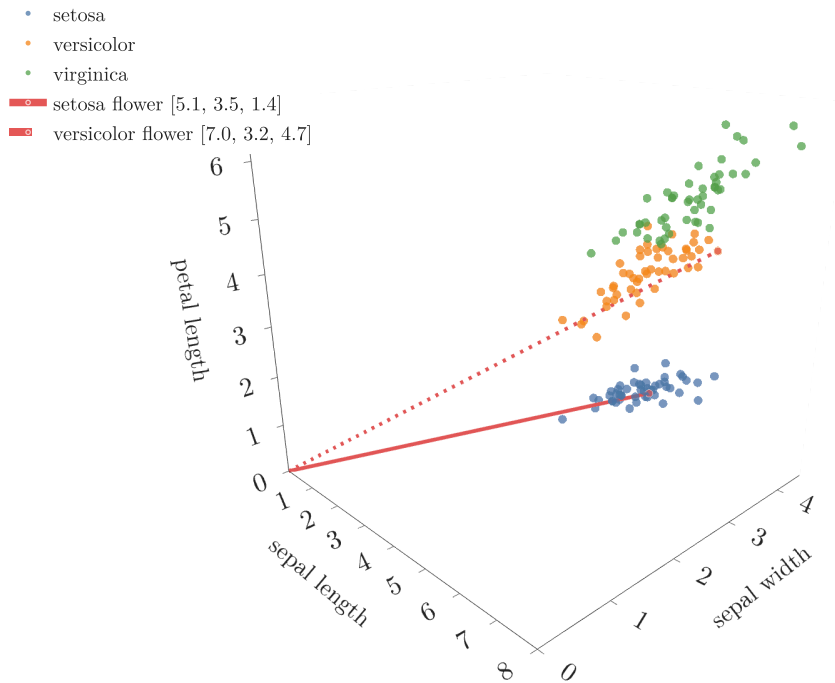


Figure 2: The iris flowers as feature vectors, using three of the four input columns

We cannot draw 4D, so Figure 2 drops petal width and plots the other three columns. Each of the 150 flowers is now a point in 3D, that is, a vector; two of them are drawn as arrows from the origin. Flowers of the same species sit close together, which is what lets a model tell the species apart.

5.2 Feature vectors hold only numbers

Key point: Text values such as “male” or “S” must be encoded as numbers before they can go into a feature vector.

The same works for any table. In the Titanic dataset, a passenger could be described by age, fare, sex, passenger class and port of embarkation: a 5-dimensional feature vector. But a feature vector is a collection of numbers; it cannot hold words like “male” or “S”.

So we encode the text values as numbers. Say male = 0 and female = 1, and the ports Q, C and S are 0, 1 and 2. A 17-year-old male passenger who paid a fare of 52 in first class and embarked at S becomes

$$x = [17, 52, 0, 1, 2]$$

The encodings themselves are taught in the [ordinal and label encoding Note](#) and the [one-hot encoding Note](#).

Extra: Coding Q, C and S as 0, 1 and 2 tells a model that S is “twice” C, which means nothing. For a column with no natural order, one-hot encoding is usually the safer choice.

6. Text as vectors: a movie recommender

Key point: Bag of words turns each text into a vector of word counts; similar texts become nearby vectors, so “recommend a similar movie” becomes “find the nearest vector”.

6.1 The problem

Key point: We have only movie names and summaries, and must recommend movies similar to the one a user picks.

A recommender system suggests items a user is likely to like. Suppose a user names a movie, say *The Matrix*, and we must recommend five similar movies, as streaming services do. Our data has just two columns: the movie’s name and a short summary of its plot.

The summaries are text, and ML algorithms do not work with text. So we must turn every summary into numbers: into a feature vector.

6.2 Bag of words

Key point: List every unique word (the vocabulary); each text becomes a vector with one count per vocabulary word.

There are many techniques for this in **NLP** (natural language processing, the part of ML that works with text). The simplest is **bag of words**:

1. Collect the vocabulary (see the [tensors Note](#)): every unique word in all the texts.
2. Give each vocabulary word one dimension.
3. Turn each text into a vector that counts how often each vocabulary word appears in it.

Take three toy “summaries”: A = *hi how are you*, B = *my name is riya*, C = *this is 2023*. Their vocabulary has 10 unique words, so each summary becomes a 10-dimensional vector (Figure 3). We cannot draw 10 dimensions, but we can imagine them the same way we imagine 3D.

Bag of words: text to feature vectors

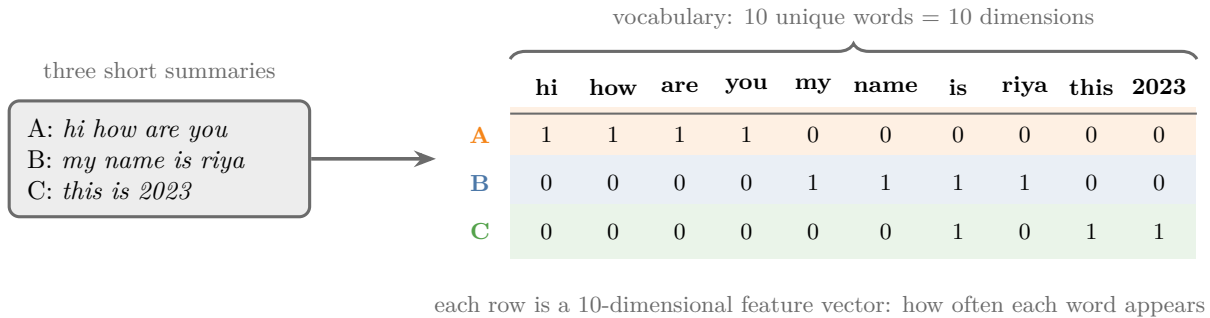


Figure 3: Bag of words turns three texts into three 10-dimensional feature vectors

Real data is much bigger. With 5,000 movies the vocabulary might hold 50,000 words: we get 5,000 feature vectors, one per movie, each a vector in a 50,000-dimensional space. The idea is exactly the same.

Extra: The order of the words is thrown away, which is why the method is called a “bag” of words: *dog bites man* and *man bites dog* get the same vector. The [tensors Note](#) (section 6.2) shows a different method, one-hot vectors per word, which keeps the order.

6.3 Recommending by distance

Key point: Movies whose vectors lie close together have similar summaries, so we recommend the nearest ones.

Now every movie is a point in the same space. If a user likes movie B, we recommend the movie whose vector lies closest to B. Using the straight-line (Euclidean) distance from the [KNN imputer Note](#):

1. **In words:** count the words that appear in one text but not the other; each adds $1^2 = 1$ to the sum of squares; take the square root.

2. **Formula:**

$$d(u, v) = \sqrt{(u_1 - v_1)^2 + \cdots + (u_{10} - v_{10})^2}$$

3. **Example:** B and C differ in *my*, *name*, *riya*, *this* and *2023* (they share *is*), so

$$d(B, C) = \sqrt{5} \approx 2.24$$

A and B share no word, so they differ in all 8 of their words:

$$d(A, B) = \sqrt{8} \approx 2.83$$

C is closer to B than A is, so a user who likes B gets C. Many real recommender systems work on exactly this idea: turn items into high-dimensional vectors, compute distances, recommend the nearest. The [dot product and cosine similarity Note](#) gives a better measure of closeness for text.

Python: Bag of words in scikit-learn.

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
texts = ["hi how are you", "my name is riya", "this is 2023"]
```

```
cv = CountVectorizer()
```

```
X = cv.fit_transform(texts).toarray()
```

```
print(cv.get_feature_names_out())    # the vocabulary
```

```
print(X)                             # one row per text
```

CountVectorizer sorts the vocabulary alphabetically, so its columns come in a different order from Figure 3; the distances do not change. The Notebook for this Note (notebook.ipynb) runs the whole recommender.

7. Row vectors and column vectors

Key point: The same vector can be written as a row (shape $1 \times n$) or as a column (shape $n \times 1$); when nothing is said, a vector means a column.

A vector can be written in two ways. The components are the same; only the layout differs.

- A **row vector** writes the components side by side. Its shape is $1 \times n$: one row, n columns.

$$a = \begin{bmatrix} a_1 & a_2 & \cdots & a_n \end{bmatrix}$$

- A **column vector** writes them one below the other. Its shape is $n \times 1$: n rows, one column.

$$b = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

Seeing the shape $1 \times n$ or $n \times 1$, we know at once which kind a vector is.

Both appear in the iris table. One flower's feature vector, $[5.1, 3.5, 1.4, 0.2]$, is a row of the table: a row vector of shape 1×4 . One whole column of the table, such as the species of all 150 flowers, is a column vector of shape 150×1 . In short: a row of the table is a row vector, a column of the table is a column vector.

In ML we use either form, depending on what a calculation needs. When a book or a formula says “vector” without saying which, it means a column vector. Turning a column into a row is called the transpose (see the [PCA step by step Note](#)); it becomes important for the dot product in the [dot product and cosine similarity Note](#).

Python: NumPy's plain 1D array has no row or column form; its shape is just $(n,)$. To make the form explicit, give it two axes.

```
import numpy as np
```

```
x = np.array([5.1, 3.5, 1.4, 0.2])    # shape (4,)
```

```
row = x.reshape(1, -1)           # shape (1, 4)
col = x.reshape(-1, 1)          # shape (4, 1)
-1 tells NumPy to work out that size itself.
```

8. Summary

Idea	What it is	Example
Vector	A point in a coordinate system	$[3, 4]$
Component	One number of a vector	3 (x-component)
Dimension	Number of components	$[3, 4]$ has dimension 2
Feature vector	The input values of one data point	$[5.1, 3.5, 1.4, 0.2]$
Bag of words	Text as word counts over a vocabulary	<i>this is 2023</i> $\rightarrow$ 10-dimensional vector
Row vector	Components side by side, shape $1 \times n$	one flower
Column vector	Components stacked, shape $n \times 1$	one column of the table

- Linear algebra matters for ML because it works in any number of dimensions and can represent any data as numbers.
- A model takes feature vectors in and gives predictions out; feature vectors hold only numbers, so text must be encoded first.
- Similar items have nearby vectors, which is the basis of many recommender systems.

9. Key terms

Term	Meaning
Linear algebra	The branch of mathematics that studies linear equations, vectors and matrices
Vector (geometric view)	A point in a coordinate system, drawn as an arrow from the origin
Component	One number of a vector, its position along one axis
Dimension of a vector	The dimension of the space it lives in: its number of components
Feature vector	The vector of input values of one data point
Recommender system	A system that suggests items a user is likely to like
NLP	Natural language processing: ML on text
Bag of words	Turning a text into a vector of word counts over the vocabulary
Row vector	A vector written as one row, shape $1 \times n$
Column vector	A vector written as one column, shape $n \times 1$; the default meaning of “vector”